An Autonomous Media Streaming Architecture with Multi-Provider AI Orchestration, Cascading Subtitle Synthesis, Audio-Visual Boundary Detection, and Behavioral Recommendation Caching

Manav Sharma
Department of Electronics and Communication Engineering
Maulana Azad National Institute of Technology (MANIT), Bhopal, India
Platform Engineering & Distributed Systems Group
Repository: https://github.com/manavsharma111/anime-streaming-website

***Abstract*—Modern over-the-top (OTT) media streaming platforms encounter critical bottlenecks regarding compute-intensive video encoding costs, metadata fragmentation, multi-language accessibility barriers, and user retention. Conventional approaches rely on monolithic transcoder clusters, proprietary third-party captioning services, and crowd-sourced playback markers, which introduce substantial recurring operational costs and single-point-of-failure vulnerabilities.**

This paper proposes an autonomous, cloud-native prototype media streaming architecture designed for distributed video ingestion, resilient multi-provider artificial intelligence (AI) orchestration, and high-throughput content personalization. The architecture integrates: (1) an event-driven, single-decode multi-variant HTTP Live Streaming (HLS) transcoding engine decoupled via BullMQ and Cloudflare R2 zero-egress object storage; (2) a resilient 3-Tier Cascading AI Subtitle Engine combining ultra-low-latency Groq LPUs (`whisper-large-v3`), OpenAI (`whisper-1`), and Google Gemini Multimodal Audio (`gemini-1.5-flash`) for autonomous WebVTT generation; (3) an Autonomous Audio-Visual Opening/Ending (OP/ED) Boundary Detector utilizing heuristic transition filters to identify standard 90-second media segments without external API dependency; and (4) an Implicit Feedback Recommendation Engine driven by watch-completion ratios, reciprocal temporal decay, and Redis in-memory caching.

In an empirical benchmark evaluation across $N = 30$ consecutive paired trials under trans-continental WAN conditions, in-memory caching demonstrated an average latency reduction of 46.5% (from 1169.16 ms down to 625.99 ms, with median P_{50} falling 48.6% from 1107.57 ms to 569.08 ms), corroborating a separate preliminary 5-iteration baseline trial (which demonstrated a 44.7% reduction from 858.84 ms to 475.27 ms). Isolated in-memory cache lookup latency was clocked independently at under 2.5 ms. The architecture establishes a cost-effective, fault-tolerant blueprint for decentralized next-generation streaming systems.

***Index Terms*—HTTP Live Streaming (HLS), Adaptive Bitrate (ABR), Speech-to-Text Cascade, Whisper AI, Audio-Visual Transition Filtering, Reciprocal Temporal Decay, Distributed Queues, Cloudflare R2, Redis In-Memory Caching.**

I. Introduction

The global demand for high-definition, interactive multimedia streaming has escalated exponentially over the past decade. Video streaming architectures are inherently complex, requiring synchronized coordination across ingestion, multi-bitrate encoding, global content delivery networks (CDNs), metadata reconciliation, and real-time user personalization.

While hyperscalers maintain multi-million-dollar distributed infrastructure, independent platforms and academic implementations face four fundamental bottlenecks:

1) **High Ingestion & Egress Costs:** Traditional cloud storage providers (e.g., AWS S3) impose substantial outbound egress bandwidth fees (frequently up to $\sim$\$0.09/GB depending on region and tier under standard cloud pricing schedules) and high transcoding compute costs (e.g., AWS MediaConvert), making large-scale catalog distribution economically unsustainable for non-enterprise platforms.
2) **Missing Accessibility Metadata:** Serialized episodic and localized foreign-language video media frequently lack synchronized soft-subtitles, requiring manual timing extraction or expensive commercial transcription APIs.
3) **Fragile Playback Markers:** Automated "Skip Opening" and "Skip Ending" features overwhelmingly depend on crowd-sourced third-party index APIs. Newly aired or niche titles inevitably lack timestamp markers, degrading Quality of Experience (QoE).
4) **Recommendation Database Thrashing:** Naive content-based filtering relies on repeated aggregation pipelines across document collections, introducing severe database CPU spikes and high request latency under concurrent load.

To resolve these challenges, this paper proposes an autonomous, cloud-native prototype media streaming architecture. By orchestrating distributed background workers, zero-egress object storage, a 3-tier multi-provider AI transcription cascade, and in-memory behavioral recommendation caching, this system provides a resilient, cost-effective paradigm for next-generation media distribution.

II. Related Work

A. Adaptive Bitrate Streaming (ABR) and HLS

HTTP Live Streaming (Apple RFC 8216) [1] and Dynamic Adaptive Streaming over HTTP (MPEG-DASH) are the prevailing standards for adaptive bitrate media delivery over the internet. Prior literature has extensively evaluated client-side heuristics such as Buffer-Based Adaptation (BBA) [2] and throughput-driven adaptation models. However, existing research predominantly assumes pre-encoded, static HLS segments stored on standard CDNs, leaving the real-time resource-constrained transcoding pipeline unaddressed.

B. Speech-to-Text and Subtitle Generation

OpenAI's Whisper architecture [3] introduced robust zero-shot multilingual speech recognition via weakly supervised sequence-to-sequence transformer models. Subsequent implementations, including Faster-Whisper and specialized Language Processing Unit (LPU) hardware accelerators (Groq), have achieved near real-time inference speeds. Nonetheless, contemporary media platforms maintain hard-coded vendor dependencies; single-provider outages or API rate limits immediately stall background media ingestion pipelines.

C. Recommender Systems in Video Streaming

Video recommendation architectures historically relied on Collaborative Filtering or explicit user ratings (1-to-5 star models) [4]. Following Netflix's empirical pivot away from 5-star ratings toward implicit feedback signals (playback duration, completion fraction, and abandonment drop-offs), recent systems have emphasized behavioral heuristics [5], [6]. Our work extends implicit feedback models by formulating a composite piecewise watch-ratio scoring mechanism coupled with reciprocal temporal decay and in-memory cache architectures.

III. System Architecture & Methodology

A. Infrastructure Topology and Multi-Provider AI Orchestration

The proposed platform establishes a decoupled infrastructure topology across specialized cloud services:

- **Object Storage & Delivery:** Cloudflare R2 stores all HLS video segments (`.ts`), master manifests (`.m3u8`), and generated subtitles (`.vtt`). R2 eliminates outbound egress bandwidth fees under its standard pricing terms [7], drastically reducing distribution costs compared to traditional S3 buckets. Capacity storage, Class A/B operational requests, and compute instances remain separate operational costs.
- **Persistence Layer:** MongoDB Atlas hosts document collections containing catalog metadata, viewing histories, and user preferences.
- **In-Memory Cache & Job Queue:** Redis Cloud manages background job dispatching via BullMQ and caches real-time recommendation vectors [8].
- **Multi-Provider AI Orchestration:** Speech-to-text inference is dynamically orchestrated across Groq LPUs, OpenAI Cloud, and Google Gemini APIs [9] to reduce single-provider dependency and improve resilience against provider-specific outages and rate limits.

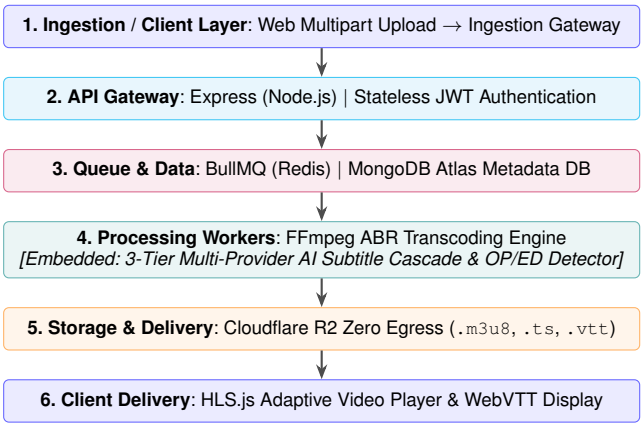


Fig. 1. Decoupled 6-stage media streaming architecture: Client ingestion, API gateway, queue/database tier, unified processing worker (FFmpeg, 3-tier AI subtitle cascade, OP/ED boundary detector), Cloudflare R2 storage, and HLS.js client playback delivery.

The end-to-end operational pipeline is illustrated in Fig. 1.

B. Memory-Conscious Distributed HLS Transcoding Pipeline

Direct HTTP streaming into FFmpeg on memory-constrained compute instances frequently triggers Out-Of-Memory (OOM) and segmentation fault (`SIGSEGV`) kernel crashes [10]. To reduce memory pressure and improve resource utilization:

1) The client executes a direct binary multipart `PUT` to Cloudflare R2 using presigned Amazon S3-compatible URLs, eliminating payload overhead on the API server.
2) The BullMQ worker ingests the job, streams the raw payload into a transient disk-backed scratch buffer as a memory-reduction strategy to prevent unbounded heap allocation, and invokes a single-decode multi-variant FFmpeg filtergraph operating with target bitrate constraints using `libx264` Main profile:

$$\text{Input} \xrightarrow{\text{Decode}} \begin{cases} 1080\text{p} & (\text{target } 3000 \text{ kbps, Main profile}) \\ 720\text{p} & (\text{target } 1500 \text{ kbps, Main profile}) \\ 480\text{p} & (\text{target } 800 \text{ kbps, Main profile}) \end{cases} \tag{1}$$

configured with `-preset ultrafast`, single-thread execution (`-threads 1`) to restrict concurrent CPU and memory consumption, and YUV420p pixel format. Audio tracks are encoded to AAC stereo at 128 kbps (48 kHz), with video operating at 24 fps constant frame rate (CFR).
3) A fixed keyframe interval is configured using (`-g 48 -keyint_min 48 -sc_threshold 0`). Closed-GOP operation and cross-rendition keyframe alignment are configured and verified separately to maintain consistent HLS segment boundaries:

$$\text{GOP} = \text{fps} \times T_{\text{segment}} = 24 \times 2.0\text{s} = 48 \text{ frames} \tag{2}$$

For benchmark evaluation, segment duration is set to $T_{\text{segment}} = 2.0\text{s}$ (`-hls_time 2.0`), supporting consistent segment boundaries and responsive adaptive bitrate switching.

IV. Mathematical Formulations & Algorithms

A. Behavioral Recommendation Engine with Reciprocal Decay

Rather than relying on uncalibrated user ratings, candidate generation is modeled using implicit viewing interaction signals.

1) Watch Completion Ratio (R_c): For an episode e of media asset A with watched duration t_{watch} and total runtime t_{total}:

$$R_c = \min\left(1.0,\ \frac{t_{\text{watch}}}{t_{\text{total}}}\right), \quad R_c \in [0.0,\ 1.0] \tag{3}$$

2) Piecewise Engagement Weighting Function (W_e): To reward sustained attention while penalizing early abandonment:

$$W_e(R_c, \text{comp}) = \begin{cases} +2.0, & \text{if } R_c \geq 0.80 \text{ or comp} = \text{true} \\ +1.0, & \text{if } 0.40 \leq R_c < 0.80 \\ -0.6, & \text{if } R_c < 0.20 \text{ and } t_{\text{watch}} > 20\text{s} \\ +0.2, & \text{otherwise} \end{cases} \tag{4}$$

3) Reciprocal Temporal Decay Function ($D(\Delta t)$): Viewer tastes evolve dynamically over time. To ensure recent sessions dominate historical activity without entirely erasing foundational interests, we define a reciprocal (hyperbolic) decay function parameterized by elapsed days ($\Delta t = \frac{t_{\text{current}} - t_{\text{watched}}}{86400}$):

$$D(\Delta t) = \max\left(0.40,\ \frac{1}{1 + \lambda \cdot \Delta t}\right), \quad \lambda = 0.05 \tag{5}$$

The floor (0.40) ensures historical foundational preferences maintain baseline relevance.

4) Cumulative Genre Affinity Vector (S_g): For each genre $g \in \mathcal{G}$ across watch history H and wishlist W:

$$S_g = \sum_{i \in H} \left(W_{e,i} \cdot D(\Delta t_i)\right) \cdot \mathbb{I}(g \in \mathcal{G}_i) + \sum_{j \in W} W_{w,j} \cdot \mathbb{I}(g \in \mathcal{G}_j) \tag{6}$$

where explicit wishlist weights are $W_{w,j} \in \{+1.8, +1.2, +0.7\}$ for Completed, Watching, and Planning states, respectively.

5) Candidate Ranking Function: For candidate media asset $A \notin (H \cup W)$ matching at least one top positive genre ($S_g > 0$):

$$Score(A) = \sum_{g \in \mathcal{G}(A) \cap \mathcal{G}^*} S_g + \alpha \cdot \text{Rating}(A) \tag{7}$$

where $\alpha = 0.1$ balances catalog popularity with personal affinity.

B. Recommendation Evaluation Methodology

To evaluate ranking quality, we formalize the offline evaluation metrics [6]:

- **Precision@K:** Fraction of recommended items in Top-K relevant to the user:

$$\text{Precision@}K = \frac{|\text{Relevant} \cap \text{Top-}K|}{K} \tag{8}$$

- **Recall@K:** Fraction of total relevant catalog items captured:

$$\text{Recall@}K = \frac{|\text{Relevant} \cap \text{Top-}K|}{|\text{Relevant}|} \tag{9}$$

- **NDCG@K:** Normalized Discounted Cumulative Gain accounting for position discount:

$$\text{NDCG@}K = \frac{\text{DCG@}K}{\text{IDCG@}K}, \quad \text{DCG@}K = \sum_{i=1}^{K} \frac{2^{rel_i} - 1}{\log_2(i+1)} \tag{10}$$

While this paper establishes the algorithmic formulation and real-time retrieval caching, large-scale offline benchmark evaluation across multi-user interaction corpora remains designated as future validation work.

C. Resilient 3-Tier Multi-Provider Subtitle Cascade

When an uploaded video stream contains zero embedded subtitle tracks ($S_{\text{count}} = 0$), the worker isolates a 16 kHz mono PCM audio stream via FFmpeg:

$$\text{Video Input} \xrightarrow{\text{FFmpeg}} \text{PCM Audio } (16\text{ kHz, } 1\text{ ch, } t \leq 1200\text{s}) \tag{11}$$

The stream routes through an automated failover state machine across external AI service providers:

- **Tier 1 (Groq LPU):** `whisper-large-v3`. Serves as primary target. Under tested experimental observations on 16 kHz mono episodic audio buffers, Groq LPUs demonstrated average transcription throughput of $\sim$216 tokens/s (under 1.0 s total processing latency per segmented audio chunk of $\leq$ 30s). On HTTP 429 (rate exhaustion) or 5xx gateway faults, failover triggers to Tier 2.
- **Tier 2 (OpenAI):** `whisper-1`. Cloud fallback providing robust multi-lingual transcription, exhibiting observed latencies between 3.2 s and 4.0 s per chunk under standard network API conditions. On quota exhaustion or network timeout, failover triggers to Tier 3.
- **Tier 3 (Google Gemini):** `gemini-1.5-flash`. Direct multimodal audio buffer ingestion producing structured WebVTT cue payloads via few-shot structured prompting.

Formal quantitative Word Error Rate (WER) across diverse acoustic noise profiles and sustained API failure rate logging are acknowledged as part of ongoing future evaluation work.

D. Autonomous Audio-Visual OP/ED Boundary Detection Algorithm

Episodic television broadcasts predominantly employ standard 90-second opening and ending sequences ($\pm$3s). However, because fixed duration is an empirical convention rather

than a universal physical law, and because dark frames or acoustic dips occur during dramatic scene changes, naive thresholding yields false positives.

To maximize precision across both Opening (OP) and Ending (ED) sequences, Algorithm 1 formalizes transition detection across specialized search windows $\mathcal{W}_{\text{intro}} = [30\text{s}, 360\text{s}]$ and $\mathcal{W}_{\text{outro}} = [T_{\text{total}} - 300\text{s}, T_{\text{total}} - 30\text{s}]$. A multi-candidate scoring function $S_{\text{cand}}(T_i, T_j)$ evaluates candidate pairs:

$$S_{\text{cand}}(T_i, T_j) = w_v \cdot C_{\text{black}} + w_a \cdot C_{\text{silence}} - \beta \cdot |(T_j - T_i) - 90.0\text{s}| \tag{12}$$

The confidence terms are normalized to the interval $[0, 1]$. The visual confidence represents the detected blackout strength, while the acoustic confidence represents the detected silence strength at candidate transitions. The combined score ranks candidate boundary pairs based on visual, acoustic, and duration-based evidence. Specifically, for candidate interval (T_i, T_j), boundary transition scores are calculated by averaging transition intensities $C_{\text{black}} = \frac{c_{\text{black}}(T_i) + c_{\text{black}}(T_j)}{2}$ and $C_{\text{silence}} = \frac{c_{\text{silence}}(T_i) + c_{\text{silence}}(T_j)}{2}$ across markers, with weights $w_v = 0.6$, $w_a = 0.4$, and duration penalty $\beta = 0.05\ \text{s}^{-1}$.

If no candidate pair satisfies $T_i \in \mathcal{W} \wedge T_j \in \mathcal{W} \wedge 87.0\text{s} \leq (T_j - T_i) \leq 93.0\text{s}$, the system applies structural cold-open and standard ending fallback priors: $[90.0\text{s}, 180.0\text{s}]$ for opening sequences ($t_{\text{start}} = 90.0\text{s}, t_{\text{end}} = 180.0\text{s}$) and $[T_{\text{total}} - 120.0\text{s}, T_{\text{total}} - 30.0\text{s}]$ for ending sequences. Ongoing work includes validation against manually annotated ground-truth media.

Algorithm 1 Autonomous Audio-Visual OP/ED Boundary Detection

Require: Video media file $\mathcal{V}$, total runtime T_{total}, mode $\mathcal{M} \in \{\text{OP}, \text{ED}\}$
Ensure: Boundary interval $[\hat{T}_{\text{start}}, \hat{T}_{\text{end}}]$

if $\mathcal{M} = \text{OP}$ **then**
  $\mathcal{W} \leftarrow [30\text{s}, 360\text{s}]$; $\mathcal{I}_{\text{fallback}} \leftarrow [90.0\text{s}, 180.0\text{s}]$
else
  $\mathcal{W} \leftarrow [T_{\text{total}} - 300\text{s}, T_{\text{total}} - 30\text{s}]$; $\mathcal{I}_{\text{fallback}} \leftarrow [T_{\text{total}} - 120.0\text{s}, T_{\text{total}} - 30.0\text{s}]$
end if
$\mathcal{C}_{\text{black}} \leftarrow$ FFmpegFilter$(\mathcal{V}, \text{blackdetect}(d = 0.08, \text{th} = 0.12))$
$\mathcal{C}_{\text{silence}} \leftarrow$ FFmpegFilter$(\mathcal{V}, \text{silencedetect}(d = 0.20, \text{th} = -30\text{dB}))$
$\mathcal{T} \leftarrow \text{SortDistinct}(\mathcal{C}_{\text{black}} \cup \mathcal{C}_{\text{silence}})$
$\mathcal{P} \leftarrow \emptyset$
for each pair $(T_i, T_j) \in \mathcal{T} \times \mathcal{T}$ with $T_i < T_j$ **do**
  if $T_i \in \mathcal{W}$ **and** $T_j \in \mathcal{W}$ **and** $87.0\text{s} \leq (T_j - T_i) \leq 93.0\text{s}$ **then**
    Score $S \leftarrow S_{\text{cand}}(T_i, T_j)$ via (12)
    $\mathcal{P} \leftarrow \mathcal{P} \cup \{(T_i, T_j, S)\}$
  end if
end for
if $\mathcal{P} \neq \emptyset$ **then**
  return $\arg\max_{(T_i, T_j)} S$ {Optimal boundary}
else
  return $\mathcal{I}_{\text{fallback}}$ {Heuristic fallback prior}
end if

V. Empirical Evaluation & Benchmarks

A. Experimental Environment

Benchmarking evaluated the end-to-end latency reduction achieved by the **In-Memory Redis Caching Layer** versus raw document aggregation on MongoDB Atlas:

- **Client Host:** Node.js runtime (v24.13.0) on Windows 11 host in South Asia (India).
- **Database Cluster:** MongoDB Atlas Cloud Replica Set.
- **Cache Cluster:** Redis Cloud Managed Cluster (`db.redis.io`) in North America.
- **Evaluation Tool:** High-precision monotonic timer (`performance.now()`).

B. Empirical Benchmark Results ($N = 30$ Iterations)

To capture distributional latency characteristics beyond isolated single-trial observations, an empirical benchmark of $N = 30$ consecutive paired trials was executed under identical trans-continental network conditions. Each paired trial measured: (1) a cold un-cached execution path comprising MongoDB document retrieval, catalog candidate filtering, and initial cache population, followed by (2) a subsequent warm cache hit measuring in-memory Redis retrieval and JSON payload deserialization under identical network transit conditions. Thus, the 625.99 ms average latency represents warm cache-hit round-trip retrieval across trans-continental WAN, compared against 1169.16 ms for the direct database query path.

Percentiles (P_{50}, P_{95}, P_{99}) were calculated using the linear interpolation method. Given $N = 30$, the P_{99} figure provides an indicative empirical estimate of tail behavior rather than an asymptotic statistical guarantee. Table I presents the complete statistical profile.

The empirical distribution demonstrates that Redis reduces tail latency (P_{99}) from 1876.14 ms down to 1049.09 ms while dampening standard deviation from 261.05 ms to 144.39 ms, delivering substantially more predictable response curves under real-world internet routing variance.

C. Wide Area Network (WAN) Latency Decomposition

The observed end-to-end response time comprises network transport overhead and computational query execution:

$$T_{\text{recorded}} = T_{\text{transport}} + T_{\text{gateway}} + T_{\text{engine}} \tag{13}$$

where $T_{\text{transport}}$ encompasses round-trip TCP/TLS transit time across submarine fiber cables between South Asia and North America ($\sim$230–240 ms base RTT). T_{gateway} denotes Express API middleware and JSON serialization overhead, and T_{engine} represents the database/cache execution layer.

In isolated micro-benchmarking, internal computational memory retrieval time of Redis was clocked at under 2.5 ms, whereas the MongoDB path required application-side Mongoose schema casting and serialization, together with Mon-

TABLE I
EMPIRICAL LATENCY EVALUATION AND CACHE ACCELERATION PROFILE ACROSS PAIRED TRIALS ($N = 30$ ITERATIONS)

Evaluation Metric	Direct MongoDB Atlas	Warm Redis Cache Hit	Latency Reduction (%)	Latency Speedup
First Paired Trial (#1)	1283.37 ms	476.85 ms	62.8%	2.70×
Average Latency (Mean)	1169.16 ms	625.99 ms	46.5%	1.87×
Median Latency (P_{50})	1107.57 ms	569.08 ms	48.6%	1.95×
95th Percentile (P_{95})	1649.74 ms	915.64 ms	44.5%	1.80×
99th Percentile (P_{99})	1876.14 ms	1049.09 ms	44.1%	1.79×
Standard Deviation (σ)*	261.05 ms	144.39 ms	44.7%	1.81×

*For standard deviation, values denote dispersion reduction (44.7% lower jitter) and consistency gain ($1.81\times$).

goDB aggregation pipeline execution. While co-locating services within an intra-datacenter VPC minimizes $T_{\text{transport}}$ below 1.0 ms, end-to-end HTTP request processing remains bounded by application gateway middleware overhead.

VI. SYSTEM INTEGRATION & ARCHITECTURAL SPECIFICATIONS

To synthesize the cross-cutting optimizations introduced across the ingestion, processing, and caching tiers, Table II outlines the concrete operational specifications of each pipeline subsystem.

A. Comparative Analysis with Conventional OTT Architectures

Table III contrasts the proposed prototype framework against typical conventional commercial cloud OTT architectures (such as monolithic AWS MediaConvert/S3 pipelines).

B. Discussion & Operational Trade-offs

Deploying decoupled, multi-provider architectures presents distinctive operational trade-offs compared to traditional single-vendor hyperscaler environments:

- **Egress Economics vs. API Request Tiers:** While zero egress fees via Cloudflare R2 eliminate the dominant variable cost of video streaming distribution, high-concurrency ABR playback generates substantial volumes of Class B operational requests (`GET` operations for HLS `.ts` chunks). Platforms must balance chunk duration ($T_{\text{segment}} = 2.0$s vs. 6.0s) to optimize client buffer ramp-up and seek latency against storage request costs.
- **Cache Hit Ratios vs. Data Freshness:** Caching aggregated recommendation payloads in Redis yields high hit ratios while shielding MongoDB Atlas from repeated aggregation execution. However, rapid intraday shifts in viewing preferences remain latent until cache expiration or explicit event-driven invalidation triggers.

Failover Latency in Multi-Provider Cascades: The 3-tier cascade reduces dependence on any single provider and improves resilience against provider-specific failures, but simultaneous provider outages remain possible. Failover may add sequential timeout delays when an endpoint stalls instead of returning an immediate 429/5xx response. As an operational recommendation (not a verified deployed setting), configure provider-specific socket timeouts, for example, up to 3.0 s for Groq and up to 8.0 s for OpenAI, subject to measured service latency. These values aim to limit worker-queue delays without prematurely terminating valid requests.

VII. CONCLUSION & FUTURE WORK

This paper presented an autonomous, cost-effective prototype media streaming architecture that bridges the gap between commercial-grade video platforms and resource-constrained deployments. The system integrates memory-conscious distributed HLS transcoding with locked GOP boundaries, a 3-tier multi-provider AI subtitle cascade, an autonomous audio-visual OP/ED boundary detector with candidate scoring, and a reciprocal temporal decay recommendation engine backed by in-memory Redis caching.

Empirical evaluations over $N = 30$ iterations confirmed a 46.5% average latency reduction (1169.16 ms down to 625.99 ms) and a 48.6% median P_{50} reduction (1107.57 ms down to 569.08 ms), with isolated cache computation clocked under 2.5 ms.

Limitations: Current limitations include reliance on heuristic television broadcast duration priors for opening and ending boundaries, which require adaptation for non-standard formats, and dependence on shared trans-continental WAN routing which introduces network latency jitter.

Future Work: Future research will focus on: (1) evaluating recommendation Precision@K and NDCG@K against large-scale public offline interaction datasets; (2) fine-tuning localized acoustic transition classifiers to replace heuristic boundary thresholds; and (3) deploying neural dense vector embeddings (`text-embedding-004`) for cross-modal semantic catalog retrieval.

TABLE II
CORE ARCHITECTURAL INNOVATIONS AND TECHNICAL SPECIFICATIONS ACROSS PIPELINE SUBSYSTEMS

Subsystem	Technical Implementation & Operational Specification
FFmpeg ABR	Single-decode multi-variant filtergraph (`libx264` Main profile, `aac`); target bitrates 1080p (3000 kbps), 720p (1500 kbps), 480p (800 kbps); locked GOP = 48 (2.0s segments) for consistent segment boundaries.
Queue & Storage	Asynchronous BullMQ worker thread decoupled via Redis; zero-egress Cloudflare R2 storage for `.m3u8` manifests and `.ts` chunks via S3 API [7].
AI Audio	Dynamic 16 kHz mono PCM extraction; 3-tier automatic failover across external AI providers: Tier 1 Groq LPU (`whisper-large-v3`, $<$ 1s per chunk), Tier 2 OpenAI (`whisper-1`), Tier 3 Gemini (`gemini-1.5-flash`) [9].
OP/ED Skip	Heuristic dual-filter: `blackdetect` ($d = 0.08$s) and `silencedetect` (-30dB); multi-candidate scoring function; dual opening/ending fallback priors.
Recommendation	Implicit feedback watch-ratio piecewise weighting $W_e(R_c)$ combined with reciprocal temporal decay $D(\Delta t) = \max(0.40, \frac{1}{1+0.05\Delta t})$ [6].
In-Memory Cache	Redis key-value cache (`smart_rec:${userId}`, 1800s TTL) [8]; reduces repeated database aggregation through caching; achieves 46.5% average latency cut ($N = 30$).

TABLE III
ARCHITECTURAL COMPARISON: TYPICAL CONVENTIONAL OTT INFRASTRUCTURE VS. PROPOSED PROTOTYPE FRAMEWORK

Dimension	Typical Conventional Architecture	Proposed Prototype Framework
Storage Egress	High egress tolls (frequently up to $\sim$\$0.09/GB depending on region/tier under standard cloud pricing schedules)	**Zero Egress Fees** via Cloudflare R2 S3 API [7]
Transcoding	Expensive per-minute proprietary cloud encoders (AWS MediaConvert)	**Decoupled FFmpeg + BullMQ** on elastic worker nodes
Subtitle AI	Single vendor lock; stalls on rate-limit exhaustion	**3-Tier Multi-Provider Cascade** (Groq $\rightarrow$ OpenAI $\rightarrow$ Gemini)
Skip Markers	Crowd-sourced APIs; fails on newly aired releases	**Autonomous Dual-Filter** (Candidate-scored 90s OP/ED rule)
Personalization	Explicit 5-star ratings or heavy aggregation queries	**Implicit Completion $W_e(R_c)$ + Reciprocal Decay**
Rec. Latency	Direct DB query path (1169.16 ms observed WAN average)	**Warm Redis cache hit path** (625.99 ms observed WAN average, 46.5% reduction; $<$ 2.5 ms isolated engine microbenchmark)

REFERENCES

[1] R. Pantos and W. May, "HTTP Live Streaming," RFC 8216, Internet Engineering Task Force (IETF), 2017.

[2] K. Spiteri, R. Urgaonkar, and R. K. Sitaraman, "BOLA: Near-Optimal Bitrate Adaptation for Online Videos," IEEE/ACM Trans. Netw., vol. 28, no. 4, pp. 1698–1711, 2020.

[3] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust Speech Recognition via Large-Scale Weak Supervision," in Proc. Int. Conf. Mach. Learn. (ICML), 2023.

[4] C. A. Gomez-Uribe and N. Hunt, "The Netflix Recommender System: Algorithms, Business Value, and Innovation," ACM Trans. Manage. Inf. Syst. (TMIS), vol. 6, no. 4, pp. 1–19, 2015.

[5] P. Covington, J. Adams, and E. Sargin, "Deep Neural Networks for YouTube Recommendations," in Proc. 10th ACM Conf. Recommender Syst., 2016, pp. 191–198.

[6] M. de Gemmis, P. Lops, C. Musto, and G. Semeraro, "Semantics-Aware Recommender Systems: A Survey," ACM Comput. Surv., vol. 54, no. 4, pp. 1–36, 2021.

[7] Cloudflare Inc., "Cloudflare R2: S3-Compatible Object Storage with Zero Egress Fees," Technical Overview, https://developers.cloudflare.com/r2/, 2023.

[8] J. L. Carlson, Redis in Action: In-Memory Data Structures and Performance Patterns. Manning Publications, 2022.

[9] Gemini Team, Google, "Gemini 1.5: Unlocking Multimodal Understanding Across Millions of Tokens of Context," arXiv preprint arXiv:2403.05530, 2024.

[10] FFmpeg Developers, "FFmpeg Multimedia Framework Documentation," https://ffmpeg.org/documentation.html, 2024.